

ESP32 → Server Integration

This is the contract between the ESP32 firmware and the IIoT predictive-maintenance backend.

- **Live server:** <https://api.eneguardian.app>
- **Live dashboard:** <https://eneguardian.app/dashboard/esp>

1. The only change you need on the ESP

In your `main.py`, change one line:

```
API_URL = "https://api.eneguardian.app/api/esp/ingest"
```

That's it. The payload you already send and the command you already parse in `execute_command()` match the server's contract exactly — no other firmware logic changes.

2. Endpoint contract

POST `/api/esp/ingest`

The ESP posts a sensor reading every cycle. The server stores it in InfluxDB and replies with a motor command in the same response.

Request body (`application/json`) — all sensor fields optional, send what you have:

```
{
  "temperature": 24.5,           // DHT22 ambient C
  "humidity": 48,                // DHT22 %
  "thermocouple": 62.3,          // MAX6675 motor surface C
  "vibration": 12.1,             // optional, accelerometer magnitude
  "device_id": "esp32_motor_1" // optional, defaults to esp32_motor_1
}
```

Response (200 OK):

```
{
  "action": "normal",          // one of: "stop" | "slow" | "normal"
  "speed_percent": 100,       // 0-100, feeds directly into ENA PWM
  "reason": "all sensors within limits",
  "source": "auto"           // "auto" = threshold logic, "override" = dashboard manual control
}
```

The shape matches what `execute_command(command)` already expects, so no firmware changes are needed beyond the `API_URL` swap.

3. Decision logic (server side)

The server picks `action` like this on every `/api/esp/ingest` :

Condition	Action	Speed %
Manual override active from dashboard	(override)	(override)
<code>thermocouple >= 85 C</code>	<code>stop</code>	0
<code>vibration >= 90 (1.5x warn)</code>	<code>stop</code>	0
<code>thermocouple >= 70 C</code>	<code>slow</code>	40
<code>vibration >= 60</code>	<code>slow</code>	50
otherwise	<code>normal</code>	100

Thresholds are env-tunable on the server (`ESP_TEMP_WARN` , `ESP_TEMP_CRITICAL` , `ESP_TEMP_FAILURE` , `ESP_VIB_WARN`) — let me know if 70 / 85 / 110 / 60 don't match the motor.

4. Other endpoints (you don't need them — the dashboard uses them)

Method	Path	Purpose
GET	/api/esp/status	Latest reading + current command + MTTF estimate
POST	/api/esp/command	Set manual override { "action": "stop slow normal auto" }
GET	/api/esp/history?minutes=30	Recent readings for the dashboard chart

5. Quick smoke test

From any machine that can reach the VPS:

```
curl -X POST https://api.eneguardian.app/api/esp/ingest \
  -H "Content-Type: application/json" \
  -d '{"temperature":25,"humidity":50,"thermocouple":65,"vibration":10}'
```

Expected reply:

```
{ "action": "normal", "speed_percent": 100, "reason": "all sensors within limits", "source": "auto" }
```

Then open <https://eneguardian.app/dashboard/esp> in a browser to see the live reading appear.

6. Notes

- **No auth right now.** Anyone who can reach port 8000 can post readings or send motor commands. Fine for the MVP, but don't ship to a customer site without adding a bearer token.
- **MTTF is a linear extrapolation** of the thermocouple slope over the last 10 min — placeholder until we wire up a real model.
- **Manual override persists** until the dashboard sends { "action": "auto" } to /api/esp/command (or you click AUTO in the UI).

- **Wi-Fi creds** still need to be filled in your `main.py` (`SSID` , `PASSWORD`).
- **No vibration sensor in the current code** — if you don't add an accelerometer, just drop the `vibration` field from the JSON; the server will only consider thermocouple thresholds.